Experiment 5 – Apply uninformed search on given problem.

Learning Objective: To understand and implement uninformed search algorithms such as Breadth-First Search (BFS), Depth-First Search (DFS), and Uniform Cost Search (UCS) for solving problems, and to analyse their behaviour, efficiency, and cost-based decision-making in traversing state spaces using Python.

Tools: Visual Code Studio

Theory: -

In Artificial Intelligence, search algorithms are fundamental techniques used to solve problems by exploring possible states and finding a path from an initial state to a goal state. Uninformed search algorithms, also known as blind search algorithms, operate without any domain-specific knowledge or heuristics. They rely solely on the structure of the problem and systematically explore the search space to find a solution. These algorithms are widely used in applications such as pathfinding, puzzle solving, game trees, and network traversal.

This experiment focuses on three important uninformed search techniques: Breadth-First Search (BFS), Depth-First Search (DFS), and Uniform Cost Search (UCS). Each algorithm follows a different strategy for exploring nodes and has its own advantages and limitations.

1. Breadth-First Search (BFS)

Breadth-First Search explores the search space level by level. It starts from the root node and visits all its immediate neighbors before moving to the next level of nodes. BFS uses a queue (FIFO – First In First Out) data structure to keep track of nodes to be explored.
BFS guarantees finding the shortest path in terms of the number of edges when all step costs are equal. It is complete, meaning it will always find a solution if one exists, but it can consume a large amount of memory because it stores all nodes at the current level. BFS is commonly used in shortest path problems, social networks, and web crawling.

2. Depth-First Search (DFS)

Depth-First Search explores the search space by going as deep as possible along one branch before backtracking. It uses a stack (LIFO – Last In First Out) or recursion to manage node exploration.
DFS is memory-efficient compared to BFS since it stores only a limited number of nodes at a time. However, it does not guarantee the shortest path and may get stuck in deep or infinite paths if not properly controlled. DFS is useful in applications such as maze solving, cycle detection, and topological sorting.

3. Uniform Cost Search (UCS)

Uniform Cost Search is an extension of BFS that considers the cost associated with each path. Instead of exploring nodes based on depth, UCS expands the node with the lowest cumulative cost from the start node. It uses a priority queue where nodes are prioritized based on their path cost.
UCS guarantees an optimal solution when all costs are non-negative, making it suitable for weighted graphs where different paths have different costs. However, it can be computationally expensive as it explores many nodes to ensure the least-cost path is found.

4. Comparison of BFS, DFS, and UCS

- BFS explores nodes level-wise and ensures the shortest path in unweighted graphs.
- DFS explores deep paths first and is more memory-efficient but not optimal.

- UCS considers path cost and guarantees the least-cost solution in weighted graphs.

5. Applications of Uninformed Search

Uninformed search algorithms are used in various real-world scenarios such as route finding, robotics navigation, puzzle solving (like the 8-puzzle problem), network analysis, and decision-making systems where no prior knowledge is available.
Overall, uninformed search techniques form the foundation of problem-solving in Artificial Intelligence. Understanding these algorithms helps in building more advanced and efficient search strategies.

Code & Output: -

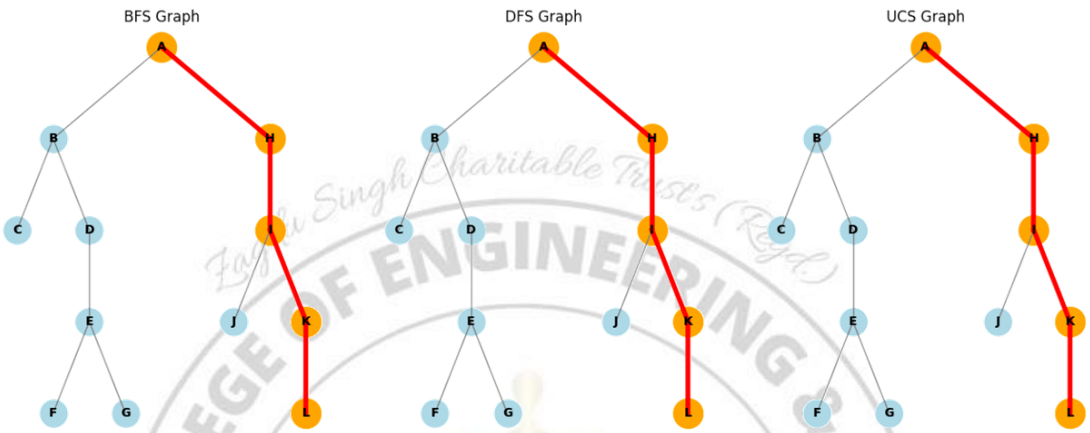
Input: -

```
import matplotlib.pyplot as plt
from collections import deque
import heapq

# -------- GRAPH --------
graph = {
    'A': ['B', 'H'],
    'B': ['C', 'D'],
    'C': [],
    'D': ['E'],
    'E': ['F', 'G'],
    'F': [],
    'G': [],
    'H': ['I'],
    'I': ['J', 'K'],
    'J': [],
    'K': ['L'],
    'L': []
}

# -------- COST GRAPH --------
cost_graph = {
    'A': [('B', 1), ('H', 1)],
    'B': [('C', 1), ('D', 1)],
    'C': [],
    'D': [('E', 1)],
    'E': [('F', 1), ('G', 1)],
    'F': [],
    'G': [],
    'H': [('I', 1)],
    'I': [('J', 1), ('K', 1)],
    'J': [],
    'K': [('L', 1)],
    'L': []
}

# -------- NODE POSITIONS --------
pos = {
    'A': (0, 5),
    'B': (-3, 4),
    'C': (-4, 3),
    'D': (-2, 3),
    'E': (-2, 2),
    'F': (-3, 1),
    'G': (-1, 1),
    'H': (3, 4),
    'I': (3, 3),
    'J': (2, 2),
    'K': (4, 2),
    'L': (4, 1)
}

# -------- DRAW FUNCTION --------
def draw_graph(ax, title, path=None):

    # Draw normal edges (light gray)
    for u in graph:
        for v in graph[u]:
            x1, y1 = pos[u]
            x2, y2 = pos[v]
            ax.plot([x1, x2], [y1, y2], color='gray',
linewidth=1)

    # Highlight path (bold red)
    if path:
        for i in range(len(path) - 1):
            x1, y1 = pos[path[i]]
            x2, y2 = pos[path[i + 1]]
```

```
        ax.plot([x1, x2], [y1, y2], color='red',
linewidth=4)

    # Draw nodes
    for node, (x, y) in pos.items():
        if path and node in path:
            ax.scatter(x, y, s=600, color='orange')  #
path nodes
        else:
            ax.scatter(x, y, s=500, color='lightblue')

        ax.text(x, y, node, ha='center', va='center',
fontweight='bold')

    ax.set_title(title)
    ax.axis('off')

# -------- BFS --------
def bfs(start, goal):
    q = deque([[start]])
    visited = set()

    while q:
        path = q.popleft()
        node = path[-1]

        if node == goal:
            return path

        if node not in visited:
            visited.add(node)
            for n in graph[node]:
                q.append(path + [n])

# -------- DFS --------
def dfs(node, goal, path, visited):
    if node == goal:
        return path

    visited.add(node)

    for n in graph[node]:
        if n not in visited:
            res = dfs(n, goal, path + [n], visited)
            if res:
                return res
    return None

# -------- UCS --------
def ucs(start, goal):
    pq = [(0, start, [start])]
    visited = set()

    while pq:
        cost, node, path = heapq.heappop(pq)

        if node == goal:
            return path, cost

        if node not in visited:
            visited.add(node)

            for neighbor, weight in cost_graph[node]:
                heapq.heappush(pq, (cost + weight,
neighbor, path + [neighbor]))

    return None, float('inf')

# -------- MAIN --------
bfs_path = bfs('A', 'L')
dfs_path = dfs('A', 'L', ['A'], set())
ucs_path, ucs_cost = ucs('A', 'L')
print("BFS Path:", bfs_path)
print("DFS Path:", dfs_path)
print("UCS Path:", ucs_path)
print("UCS Cost:", ucs_cost)

# Create 3 graphs in one window
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
draw_graph(axes[0], "BFS Graph", bfs_path)
draw_graph(axes[1], "DFS Graph", dfs_path)
draw_graph(axes[2], "UCS Graph", ucs_path)
plt.show()
```

Output:

BFS Path: ['A', 'H', 'I', 'K', 'L']
DFS Path: ['A', 'H', 'I', 'K', 'L']
UCS Path: ['A', 'H', 'I', 'K', 'L']
UCS Cost: 4



Fateh Singh Charitable Trust's (Regd.)
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY
tcet
Estd. 2001
ISO 9001 : 2015 Certified
NBA and NAAC Accredited

Learning Outcomes: The student should have the ability to:

LO 5.1: Apply Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms to traverse and explore state spaces.
LO 5.2: Implement Uniform Cost Search (UCS) to determine the least-cost path in weighted problems.
LO 5.3: Analyze and compare the performance, completeness, and optimality of uninformed search algorithms.
LO 5.4: Utilize uninformed search techniques for solving real-world problem scenarios using Python.

Course Outcomes:

Upon completion of this experiment, students will be able to understand and apply uninformed search algorithms such as BFS, DFS, and UCS to solve problem-solving tasks, and evaluate their efficiency and suitability for different types of search spaces in Artificial Intelligence.

Conclusion:

Jagdu Singh Charitable Trust's (Regd.)
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY
Estd. 2001
ISO 9001 : 2015 Certified
NBA and NAAC Accredited

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				